

TSOpt guide 5: Check, decide and plan ahead

Contents

1	Score any set, compare with the alternatives	2
2	Would it have worked in your programme? <code>tso_backtest()</code>	3
3	How sure is the advantage? <code>tso_uncertainty()</code>	4
4	The value of each plot	5
5	How many plots pay off? <code>tso_economics()</code>	6
6	After harvest: how good was it really?	7
7	Can the set find the best lines?	8
8	Accuracy against diversity: the Pareto front	9
9	Two-stage designs: where the next plots go	10
10	Planning the season across stages: <code>tso_season()</code>	12
11	Next season: <code>tso_update()</code>	12
12	Which crosses to make: <code>tso_crosses()</code>	13
13	Which parents, and how much of each	14
14	The long term	15
15	Figures for papers, slides and reports	16
16	The field list and the one-page report	18
17	Outside R scripts: the command line	18
18	Decision checklist	19

A plan is not finished when `tso_design()` returns. Before the seed goes out you want to know how it compares with what you would otherwise have planted, how sure that advantage is, what each plot is worth and how many plots pay for themselves; after harvest, how well it really predicted; and then what to do next: where the next plots go, which parents to cross, and whether the strategy holds up over several cycles. This guide walks through those steps and ends with a checklist for the decision meeting.

We use the example panel (300 lines) with its phenotypes and, because it is simulated, its true genetic values. The last 50 lines are this season's targets.

```
library(TSOpt)
ex <- function(f) system.file("extdata", f, package = "TSOpt")
```

```
tp <- tso_data(ex("example_panel.vcf.gz"), verbose = FALSE)
ids <- tp$units$id
targets <- ids[251:300]
plan <- tso_design(tp, n = 60, target = targets, verbose = FALSE)

pheno <- read.csv(ex("example_pheno.csv")) # id, y (phenotype)
truth <- read.csv(ex("example_true_values.csv")) # id, g (true value)
y <- setNames(pheno$y, pheno$id)
tv <- setNames(truth$g, truth$id)
```

1 Score any set, compare with the alternatives

`tso_score()` computes the design criteria for *any* training set: the one a colleague proposes, last year's list, a set from another package. The criteria are CDmean (mean reliability of the target predictions; higher is better), its exact version with the intercept projected out, PEVmean (mean prediction error variance; lower is better) and D-optimality (information in the set, no target).

```
set.seed(7)
colleague <- sample(setdiff(ids, targets), 60)
rbind(colleague = tso_score(tp, train = colleague, target = targets),
      optimised = tso_score(tp, train = plan$selected$id, target = targets))
#>           cdmean cdmean_exact pevmean      dopt
#> colleague 0.05341986 0.05134209 0.7709016 36.70401
#> optimised 0.22905659 0.22862238 0.6277034 37.24745
```

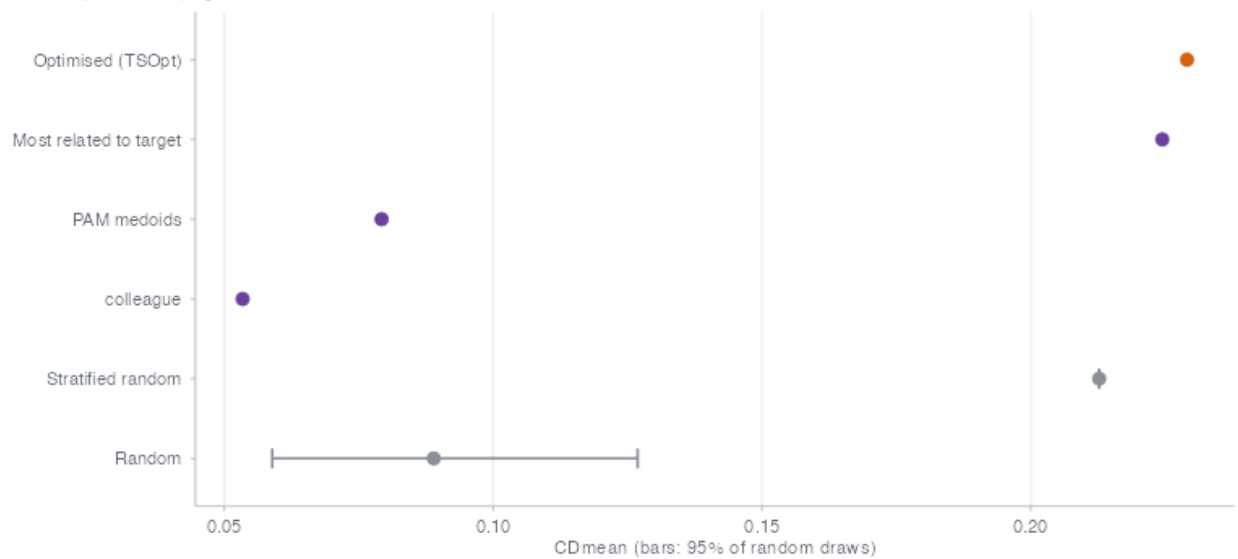
`tso_compare()` puts the plan beside the alternatives a breeder would otherwise use: random sampling (a distribution of `n_random` draws, not one lucky or unlucky set), stratified random sampling, the lines most related to the targets, partitioning around medoids (PAM), and any sets you add with others.

```
cmp <- tso_compare(plan, others = list(colleague = colleague), n_random = 30)
cmp
#> TSOpt comparison · 60 units each · target: 50 named targets
#> Method           CDmean vs random Targets >= 0.20 Unreachable
#> Optimised (TSOpt) 0.229 60.5%      100%           0
#> Random (mean)     0.089 0.0%       90%           0
#> Stratified random (mean) 0.213 54.6%      100%           0
#> Most related to target 0.224 58.8%      100%           0
#> PAM medoids       0.079 -5.6%       88%           0
#> colleague         0.053 -22.5%      72%           0
#> Unselectable
#> 0
#> 0
#> 0
#> 0
#> 0
#> 0
#> 'vs random' is on the accuracy scale (sqrt CDmean). Plot with tso_plot(comparison).
```

```
tso_plot(cmp)
```

The plan against the alternatives

60 units per method; higher is better



Here the colleague's list is a random draw from the same candidates, and an unlucky one.

What to look at: the vs random column, on the accuracy scale. The colleague's set is -23% against random, the plan 60%. If a simple rule such as stratified sampling comes close to the plan, as it does here, that is useful to know: the rule is easy to explain, and the plan's extra gain is what you pay the optimisation for.

2 Would it have worked in your programme? `tso_backtest()`

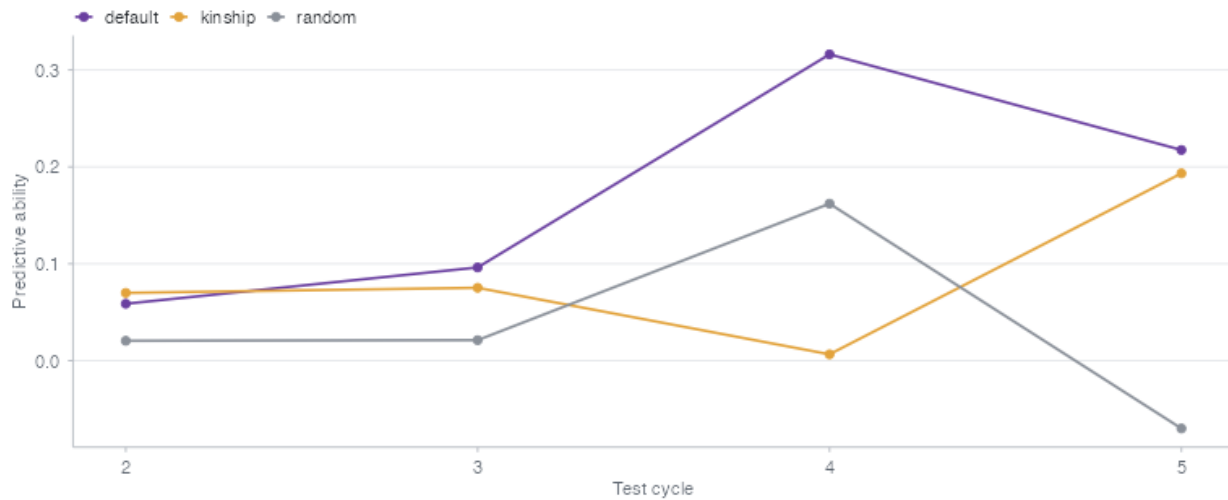
The strongest test of any method is your own history, replayed forward in time: in each past cycle, use only what was known then, choose the set, predict, and compare with what was later observed. `tso_backtest()` does exactly that on a table of `id`, `cycle` and the trait. Here on a simulated five-cycle programme (`tso_simulate_cycles()`):

```
hist <- tso_simulate_cycles(cycles = 5, n_candidates = 120, n_crosses = 12, n_parents =
  ↪ 12,
                           markers = 600, seed = 1)
bt <- tso_backtest(hist$geno, hist$pheno, n = 0.25, scheme = "from_history",
  ↪ methods = c("random", "default", "kinship"), n_random = 3, verbose =
  ↪ FALSE)

bt
#> TSOpt forward validation · from_history · 4 test cycles x 1 trait(s) · n = 0.25 per
  ↪ cycle (1.4 s)
#> method          PA   rank  top10  PA vs random (95% CI) · wins
#> random          0.033 0.043 0.160 reference
#> default          0.172 0.181 0.167 +0.139 (-0.037 to +0.314) · 100% of 4
#> kinship          0.086 0.094 0.104 +0.053 (-0.219 to +0.325) · 75% of 4
#> Predictive ability within each test cycle, from data available at the time.
  ↪ plot(bt) shows every cycle.
```

tso_plot(bt)**Forward validation: accuracy in each cycle, from the data available at the time**

from_history · n = 0.25 per cycle



Two questions, two schemes: "within_cycle" asks which of this cycle's candidates to phenotype, with every earlier cycle as training data; "from_history" asks which historical lines to train on to predict the new cycle. In 40 simulated programme histories (benchmark/forward_benchmark.R), the default beat random by +0.009 within the cycle ($t = 3.1$), because the accumulated history already carries most of the information, and by +0.091 from history ($t = 13.4$, better in 83% of cycles). The "most related to the target" heuristic was again harmful. Run it on your own history before trusting any method, this one included.

3 How sure is the advantage? tso_uncertainty()

The plan's verdict is computed on one relationship matrix, and that matrix is an estimate: another SNP chip would give a slightly different one. `tso_uncertainty()` asks whether the advantage survives. It resamples the markers with replacement, rebuilds the relationship matrix, and scores the plan and `n_random` random sets of the same size again, `B` times. The set itself is not re-optimised: it is the plan you would plant, judged under matrices it was not chosen for.

```
u <- tso_uncertainty(plan, B = 30, n_random = 10)
u
#> TSOpt uncertainty · marker bootstrap, 30 replicates of 958 markers · 10 random sets
  ↳ each (0.6 s)
#>      estimate [lower, upper] (90% intervals; estimate on all markers)
#> Plan CDmean    0.229 [0.225, 0.234]
#> Random mean    0.083 [0.072, 0.093]
#> Gain (CD)      175.2% [155.9%, 197.6%]
#> Gain (acc.)    67.1% [60.3%, 74.9%] (plan reported 61.5%)
#> Beats random   100% of replicates
#> Verdict        ROBUST: the 90% interval of the accuracy gain lies above 2%: the
  ↳ advantage holds whichever markers are used
```

What to look at: the Gain (acc.) and Verdict lines. The 90% interval of the gain in accuracy over random runs from 60% to 75%, and the plan beats random in 100% of the replicates: **ROBUST**. The verdict is robust when the whole interval is above 2%, small when it is positive but may be below 2%,

uncertain when it contains 0, not supported below 0. After an AMBER or RED plan verdict, this tells you whether the small gain is real. Use $B = 50$ (the default) or more for the final answer.

4 The value of each plot

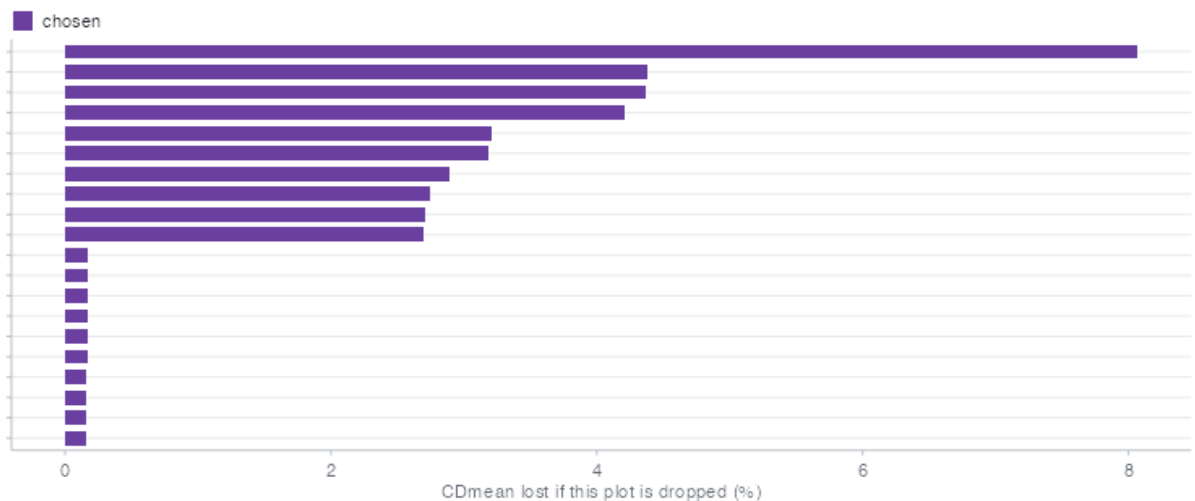
Seed runs short, a field is smaller than planned, a line fails to germinate. Which plots can you lose? `plan$selected$value` is the exact CDmean the plan loses if that one plot is dropped, and `value_pct` the same as a percentage of the plan's CDmean. The plan prints the range in its Plot value line.

```
cat(grep("^Plot value", capture.output(print(plan))), value = TRUE), "\n")
#> Plot value dropping one plot costs 0.15% to 8.07% of the CDmean (median 0.23%) · see
  ↳ tso_plot(plan, "value")
v <- plan$selected[order(-plan$selected$value), c("order", "id", "reliability",
  "value", "value_pct")]
print(rbind(head(v, 3), tail(v, 3)), digits = 3)
#>   order  id reliability  value value_pct
#> 1      1  L234      0.588 0.018446    8.068
#> 6      6  L244      0.553 0.010011    4.379
#> 4      4  L238      0.556 0.009984    4.367
#> 51     51 L116      0.560 0.000367    0.160
#> 58     58 L193      0.563 0.000364    0.159
#> 60     60 L202      0.547 0.000350    0.153
```

```
tso_plot(plan, "value", top = 20)
```

The value of each plot

the 10 most and 10 least valuable of 60 plots · drop from the bottom when seed or land runs short



What to look at: the plots are far from equal. The most valuable one, L234, carries 8.1% of the CDmean; the median plot only 0.23%. The valuable plots are mostly the first ones the search picked, typically lines with few other relatives in the set. When plots must go, drop from the bottom of this list, and give spare seed or a second plot to the top (`tso_layout(prepare = )` does this, guide 3). The values add up to only 72% of the CDmean: plots partly stand in for one another, so dropping two that cover the same targets costs more than the sum of their values. For several cuts at once, re-run `tso_design()` with the survivors in `must_include`.

5 How many plots pay off? tso_economics()

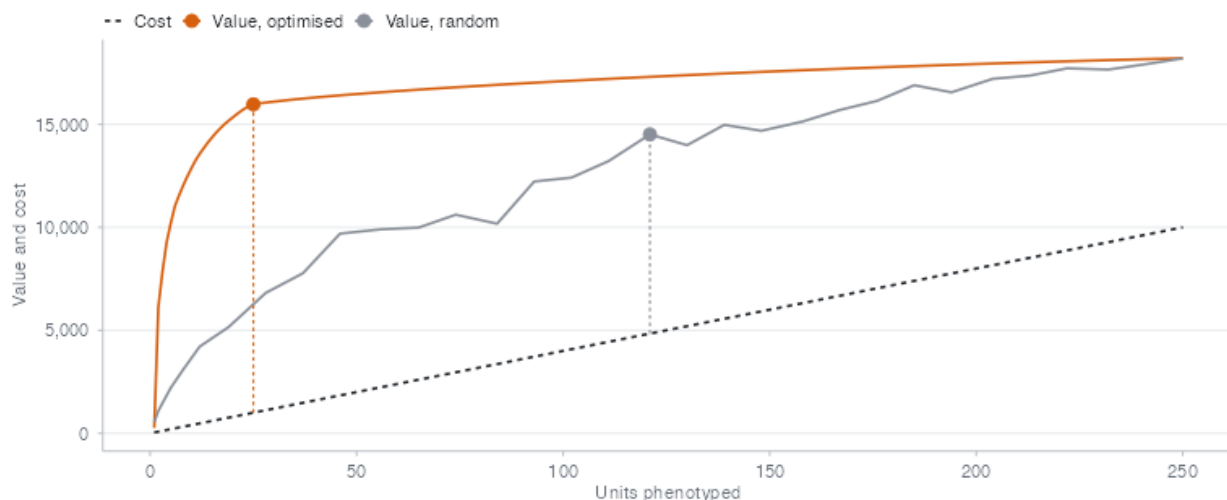
`tso_sample_size()` (guide 2) shows where extra lines stop buying accuracy. `tso_economics()` turns the same curve into money. For every size it weighs the value of the gain from selecting the top `select_top` of the targets on their predictions (`value_per_sd`: what one genetic standard deviation of gain in the selected lines is worth to you, per cycle) against `cost_per_plot`, and finds the size with the largest net value, for the optimised design and for random sampling. Here a plot costs 40 and one SD of gain is worth 20,000:

```
ec <- tso_economics(tp, cost_per_plot = 40, value_per_sd = 20000,
                    target = targets, n_random = 5)
ec
#> TSOpt economics • 250 candidates • select top 10% (i = 1.75) • averaged over h2 =
  ↳ 0.2, 0.5, 0.8
#> Prices      40 per plot • fixed 0 • 20,000 per genetic SD • 1 cycle(s)
#> → Phenotype 25 lines: net value 14,972; each extra plot now returns 30.9 against a
  ↳ cost of 40 (the last one returned 94.1)
#>
#>          n      accuracy  value      net      break-even
#>    optimised  25      0.455   15,972   14,972          1
#>    random    121      0.413   14,510    9,670          1
#> Advantage  optimised design is worth 5,302 more at its optimum and uses 96 fewer
  ↳ plots
#> Method      exact greedy accuracy curve (optimised) • mean of 5 random sets at 30
  ↳ sizes, interpolated (random)
```

`tso_plot(ec)`

Phenotype 25 lines: net value 14,972

Dashed segments: net value at each design's optimum (optimised 25, random 121) • advantage 5,302



What to look at: the arrow line and the Advantage line. The net value peaks at 25 optimised plots: the last of them returned 94.1 against its cost of 40, and one more would return only 30.9. Random sampling needs 121 plots to reach its own, lower optimum; the optimised design is worth 5,302 more and uses 96 fewer plots. The optimum depends on the prices, so run it with your own: `horizon` for a training set used over several cycles, `fixed_cost` for costs that do not depend on the size, `sigma_g` to price in trait units.

6 After harvest: how good was it really?

`tso_evaluate()` fits GBLUP on the training lines and reports how well it predicted the targets. With truth (simulations only) the metrics are computed against the true genetic values; with real data they are computed against the targets' phenotypes.

```
ev <- tso_evaluate(plan, y, truth = tv, h2 = 0.4)
ev
#> TSOpt evaluation · 60 training units -> 50 targets · against true genetic values
#> Predictive ability      0.635 (same season: optimistic; forward prediction is usually
  ↪ lower)
#> Rank correlation       0.600
#> NDCG@10%             0.763   top 10% overlap 20%
#> Realised gain         0.58 SD above the target mean (selecting the top 10%)
#> Bias slope            2.27 (1 = unbiased; > 1 = predictions shrunk)
#> Heritability used     0.40 (given)
```

What each line means:

- **Predictive ability:** the correlation between predictions and the truth (or the phenotypes). Against phenotypes it is capped by the square root of the heritability (0.63 at $h^2 = 0.4$), so judge it against that ceiling. Within one season the number is optimistic; use forward = TRUE when the targets were tested in a later season or site.
- **Rank correlation:** the same on ranks (Spearman). It is what matters for selection, and is less sensitive to a few extreme lines.
- **NDCG@10%:** how good the top of the predicted ranking is, from 0 to 1, with more weight on getting the very best lines right. **Top 10% overlap** is the plain share of the predicted top 10% that is truly top 10%.
- **Realised gain:** how far above the target mean the lines you would select (the predicted top 10%) really are, in standard deviations. This is the number to show a programme manager.
- **Bias slope:** the regression of truth on prediction. 1 means unbiased; above 1, the predictions are shrunk (spread too narrowly). It matters when predictions are used on an absolute scale, for example to set a selection threshold.

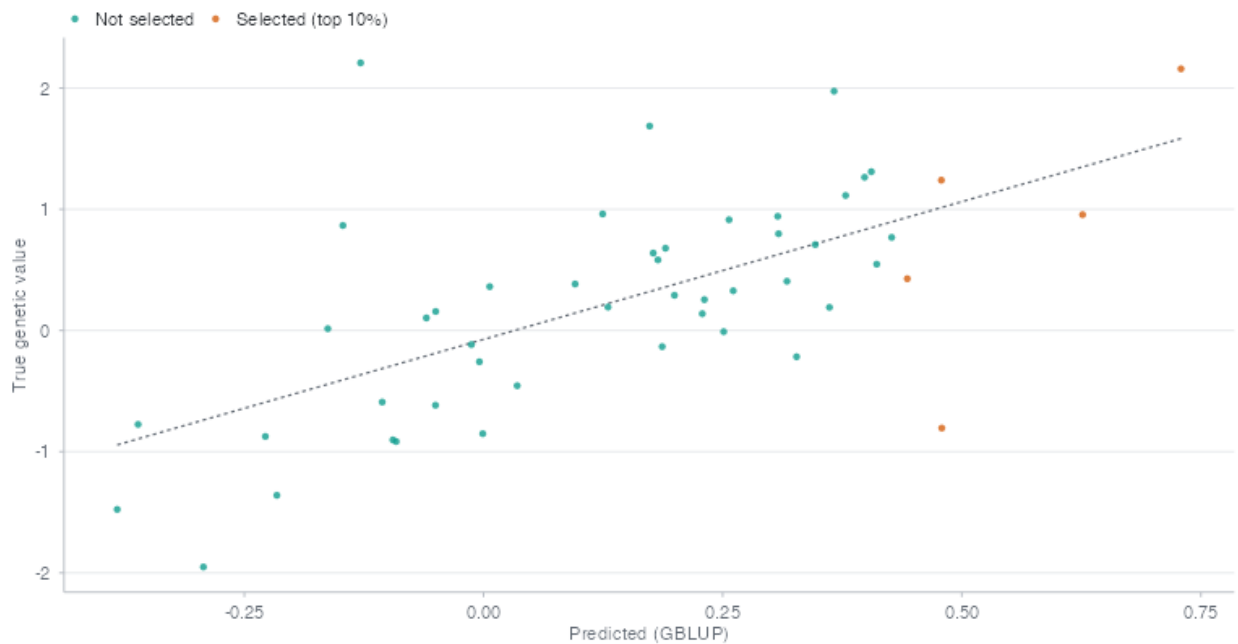
The same evaluation against phenotypes, as it would be done with real data:

```
ev_y <- tso_evaluate(plan, y, h2 = 0.4)
round(c(predictive_ability = ev_y$predictive_ability,
        realised_gain = ev_y$realised_gain), 3)
#> predictive_ability      realised_gain
#>                0.366                0.098
```

```
tso_plot(ev)
```

Predictive ability 0.64 · realised gain 0.58 SD

60 training units predicting 50 targets



Why pass h2. Without h2, TSOpt estimates the heritability by REML from the training lines. With 60 lines that estimate is unstable and often lands on its lower bound (0.02). The predictions are then shrunk almost to zero, and the bias slope and realised gain become meaningless. TSOpt warns when this happens:

```
ev_reml <- withCallingHandlers(
  tso_evaluate(plan, y, truth = tv),
  warning = function(w) {
    cat(strwrap(conditionMessage(w), 80), sep = "\n")
    invokeRestart("muffleWarning")
  })
#> REML pushed the heritability to its boundary (h2 = 0.02), which happens when
#> the training set is too small to estimate it. The bias slope and realised gain
#> are then unreliable; predictive ability and rank metrics are less affected.
#> Pass `h2 = ` (e.g. from a previous trial) to fix it.
round(c(h2_used = ev_reml$h2_used, bias_slope = ev_reml$bias_slope), 2)
#>    h2_used bias_slope
#>    0.02    48.47
```

What to look at: with the REML estimate at its boundary the bias slope is 48.5 instead of 2.27. Pass the heritability you know from earlier trials of the trait ($h^2 = 0.4$ here), and use REML only with a few hundred training lines.

7 Can the set find the best lines?

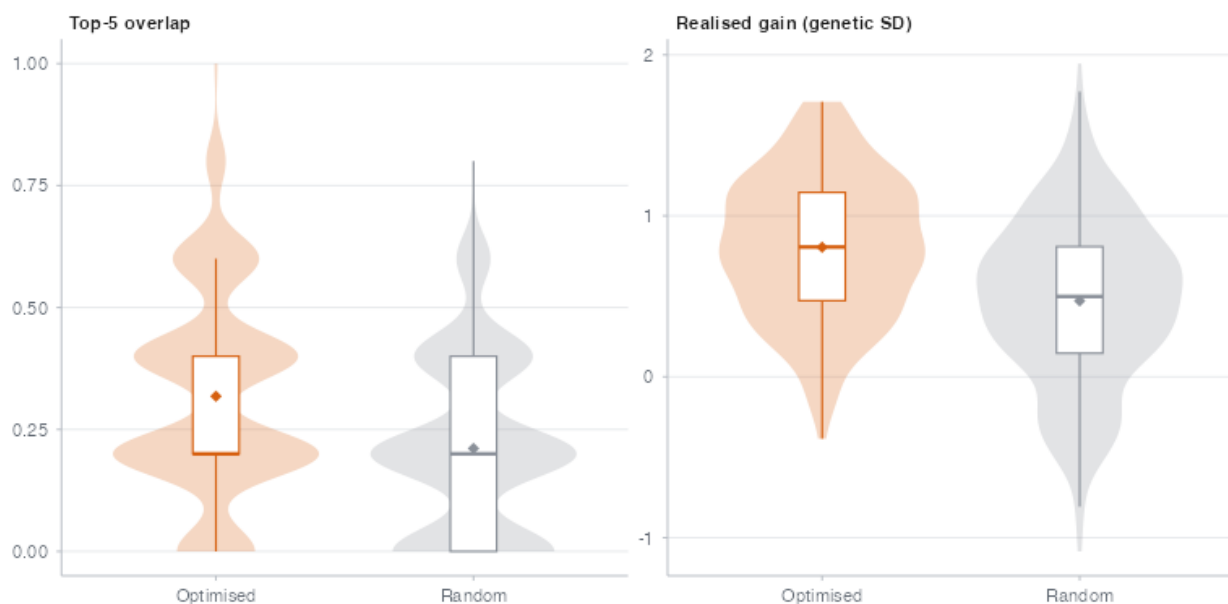
Before any phenotype exists you can still ask the question a breeder cares about: if I train on this set and select the top 10% of the targets, how many of them are truly in the top 10%? `tso_topk()` answers it by simulation: it draws genetic values from the model behind the design, simulates phenotypes on the training set, predicts, selects, and counts. It does the same for random training sets of the same size, so the answer comes with a baseline.

```
tk <- tso_topk(plan, top = 0.1, n_sim = 200, n_random = 10)
tk
#> TSOpt top-k · 60 training units → rank 50 units · select top 10% (k = 5) · h2 = 0.50
#>
#>                optimised [90% int.]      random [90% int.]
#> Top-k overlap      0.318 [0.000, 0.600]    0.211 [0.000, 0.600]
#> NDCG@5            0.756 [0.568, 0.932]    0.660 [0.413, 0.857]
#> Realised gain (SD) 0.805 [0.097, 1.515]    0.472 [-0.421, 1.261]
#> Correlation        0.483 [0.163, 0.736]    0.273 [-0.101, 0.615]
#> → Of the 5 lines you select, on average 1.6 are truly in the top 5 (random training
  ↪ sets: 1.1); chance alone gives 0.5.
#> Simulation          200 draws (Cholesky) · 10 random sets x 40 draws
```

```
tso_plot(tk)
```

Finding the top 10%: 32% overlap vs 21% for random sets

60 training units ranking 50 units · h2 = 0.50 · 200 simulated traits



What to look at: the arrow line. With 5 lines selected from 50 targets, the plan finds on average 1.6 of the true top 5, random training sets 1.1. The 90% intervals are wide: in any one season, luck matters a lot. Use more draws ($n_sim = 500$, the default) for a final answer.

8 Accuracy against diversity: the Pareto front

`tso_design()` optimises one criterion. When two goals conflict, for example accuracy (CD-mean) and a diverse, unrelated training set (low coancestry), there is no single best set. `tso_pareto()` finds the **Pareto front**: every set that no other set beats on both objectives at once. Other built-in objectives are "cdmin", "cost", "dopt", "coverage" and "gain", and you can add your own functions.

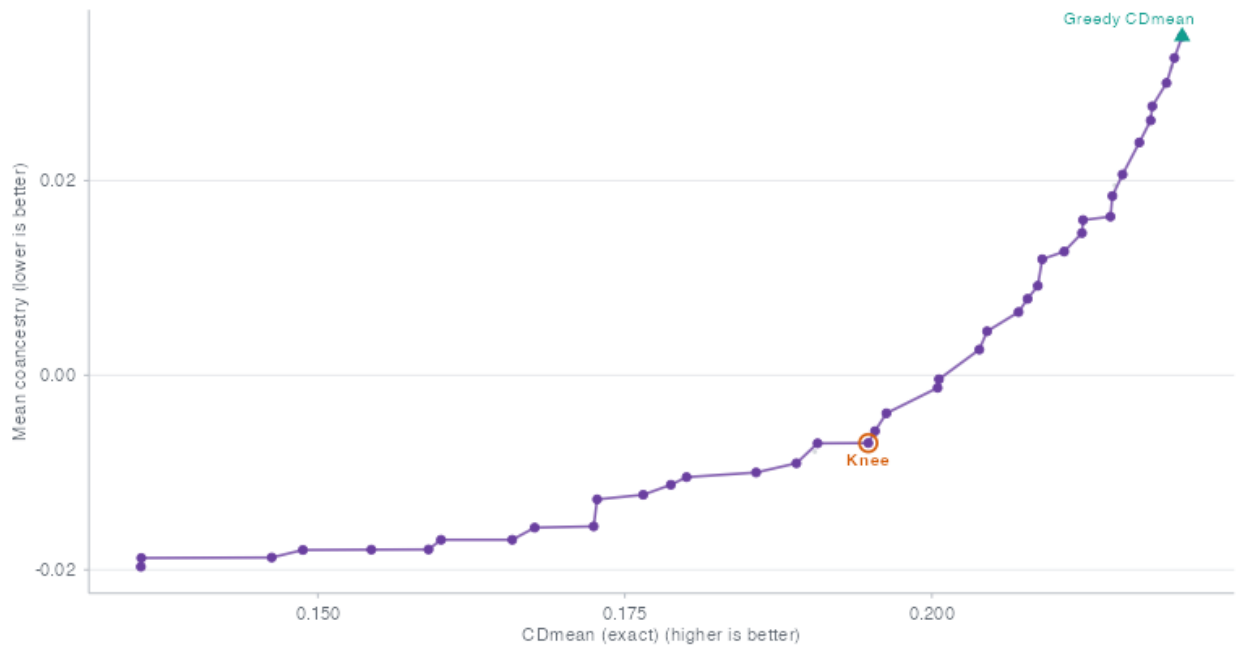
```
fr <- tso_pareto(tp, n = 40, objectives = c("cdmean", "coancestry"),
  target = targets, pop = 40, generations = 40, verbose = FALSE)
fr
#> TSOpt Pareto front · 40 non-dominated training sets of 40 from 250 candidates · 50
  ↪ named targets
```

```
#> cdmean      [max]      0.136 – 0.22 (CDmean (exact))
#> coancestry   [min]      -0.0197 – 0.0348 (Mean coancestry)
#> Knee        set 23: cdmean 0.195 · coancestry -0.00698 (farthest from the line
  ↳ between the extremes)
#> Greedy seed  cdmean 0.22 · coancestry 0.0348 (on the front)
#> Search      NSGA-II, pop 40, 40 generations, 1,647 sets evaluated (0.4 s) ·
  ↳ hypervolume 0.901
#> Next        plot(front) · tso_pareto_pick(front) · tso_pareto_pick(front,
  ↳ "best_cdmean")
```

`plot(fr)`

Pareto front: CDmean (exact) vs Mean coancestry

40 non-dominated sets of 40 (purple) · grey: final population



Take one set with `tso_pareto_pick()`. The default is the **knee**: the point where giving up a little accuracy stops buying much diversity. You can also take the best set on one objective, or weight the two.

```
knee    <- tso_pareto_pick(fr)                # the balanced choice
best_cd <- tso_pareto_pick(fr, "best_cdmean")
wtd     <- tso_pareto_pick(fr, weights = c(cdmean = 2, coancestry = 1))
round(rbind(knee = attr(knee, "values"), best_cdmean = attr(best_cd, "values"),
  weighted = attr(wtd, "values")), 4)
#>          cdmean coancestry
#> knee      0.1948   -0.0070
#> best_cdmean 0.2204    0.0348
#> weighted   0.2071    0.0065
```

9 Two-stage designs: where the next plots go

Before any phenotype exists, criteria such as CDmean cannot use data. After a first stage (a nursery, a preliminary yield trial) they can. `tso_next_batch()` updates what we know about every line's genetic value given the stage-1 phenotypes, and then asks, for each possible next

plot: *how much would this plot improve the lines I finally select?*

That quantity is the **knowledge gradient**. In plain terms: a line that is clearly in the top 10% or clearly out gains nothing from another plot, because the decision will not change. A line near the selection threshold, whose side of the line is uncertain, gains a lot, and so do its close relatives, because their data move its prediction. The batch goes to those plots.

```
nb <- tso_next_batch(plan, y[plan$selected$id], n = 25, select_top = 0.1,
                    h2 = 0.4)

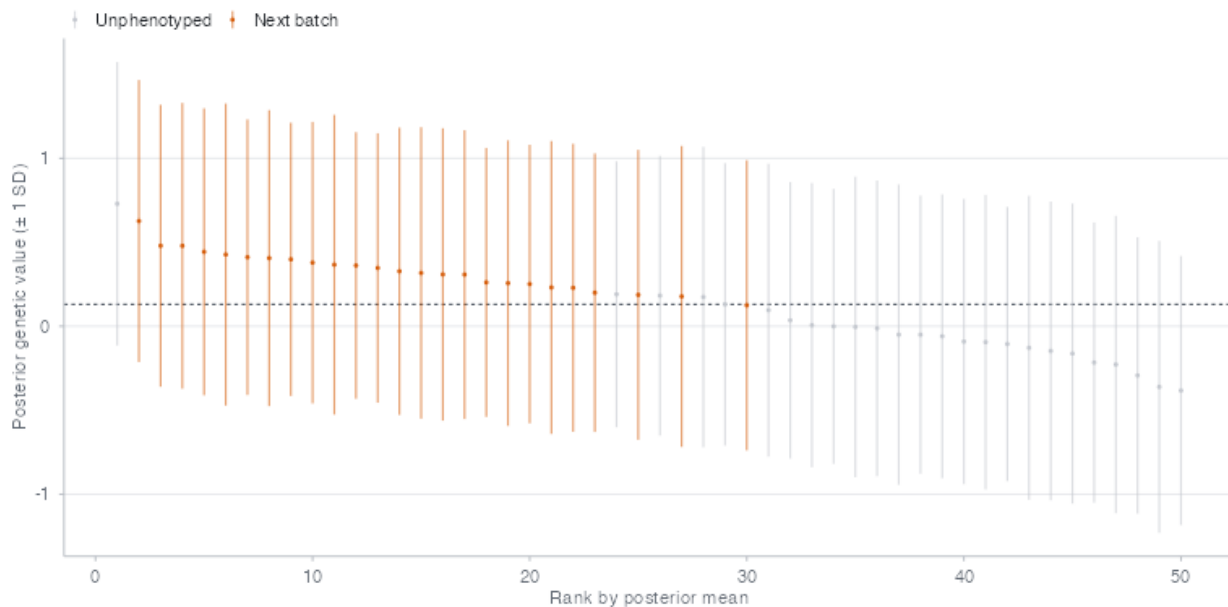
nb
#> TSOpt next batch · 25 plots · selecting the top 10% (5 of 50) · 60 stage-1
  ↳ observations
#> Posterior      h2 = 0.40 (given)
#> Batch          25 new units, 0 re-tests of promising phenotyped units
#> Boundary       24 units have a 10-90% chance of being selected: these decide where
  ↳ plots go
#> Next           tso_plot(batch) · batch$selected · batch$posterior
head(nb$posterior[, c("id", "mean", "sd", "p_top", "next_batch")], 5)
#>      id      mean      sd p_top next_batch
#> L300 L300 0.7291574 0.8439636 0.235      FALSE
#> L295 L295 0.6263055 0.8393767 0.200       TRUE
#> L282 L282 0.4790911 0.8392152 0.161       TRUE
#> L265 L265 0.4786490 0.8505572 0.164       TRUE
#> L299 L299 0.4430266 0.8539241 0.139       TRUE
```

nb\$posterior gives every line in the pool its posterior mean, standard deviation and probability of being in the top 10% (p_top): a ranking with honest uncertainty to take to the selection meeting. retest = TRUE (the default) allows a second plot on a promising line already phenotyped.

tso_plot(nb)

Where the next plots go

Dashed: selection threshold (top 10%) · plots go where the ranking is uncertain



10 Planning the season across stages: `tso_season()`

A season is more than one trial: stage 1 phenotypes a training set, genomic prediction ranks everyone, the best lines go to a multi-site stage 2, and the final choice uses all the data. Where should the plots go? `tso_season()` simulates every allocation that fits your budget many times on your panel (genotype x environment and plot error included), with the same simulated truths for every allocation:

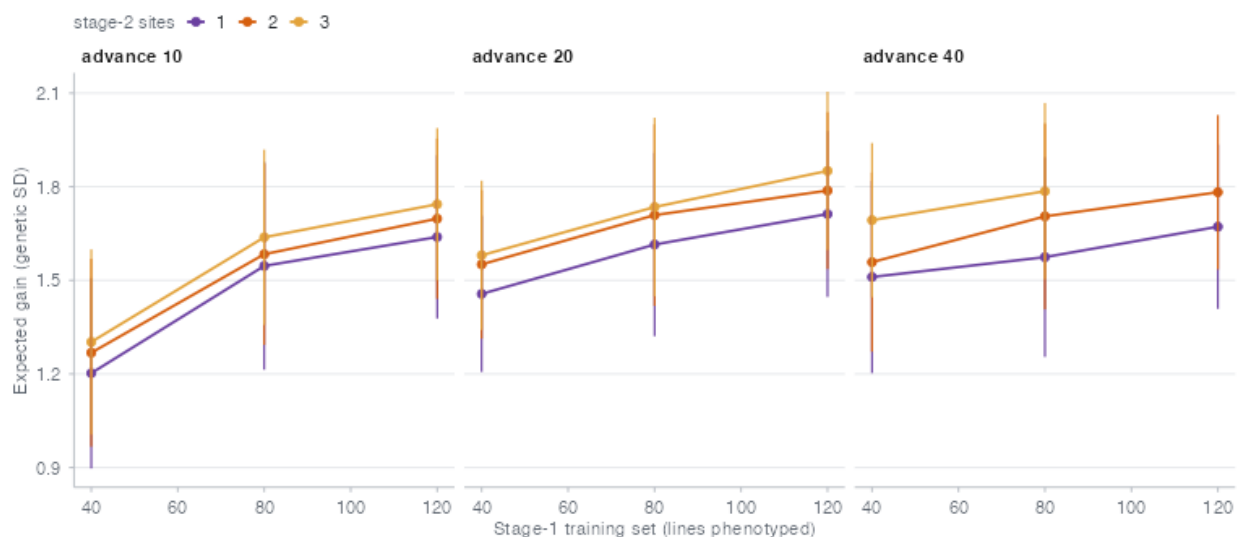
```
sp <- tso_season(tp, budget = 200, n1 = c(40, 80, 120), k1 = c(10, 20, 40), sites2 =
  ↪ 1:3,
               reps = 20, stage1 = "both", verbose = FALSE)

sp
#> TSOpt season plan · 300 candidates · budget 200 plots · 26 allocations x 20
  ↪ simulated seasons (1.5 s)
#> one plot: h2 0.30, GxE 0.50 x genetic variance · stage 1 at 1 site(s) x 1 rep(s),
  ↪ TSOpt (random alongside) training set
#> Best: stage 1 = 120 lines phenotyped (120 plots) · advance 20 to 3 site(s) (60
  ↪ plots) · select 5
#> expected gain 1.85 genetic SD (SE 0.13) · 33% of the true top 5 selected ·
  ↪ stage-1 accuracy 0.50
#> 6 other allocation(s) not clearly worse (paired difference within 2 SE; see $table)
#> TSOpt vs random stage-1 training set, at the best allocation: +0.046 genetic SD
  ↪ (SE 0.071)
```

`tso_plot(sp)`

Where the plots should go

budget 200 plots · bars: ± 2 SE over 20 simulated seasons



What to look at: allocations not clearly worse than the best (paired differences within 2 SE) are equally good choices; pick the one that is easiest to run. `stage1 = "both"` also shows what the TSOpt stage-1 design adds over random within the season.

11 Next season: `tso_update()`

`tso_update()` carries a plan forward. The lines phenotyped before become history (free, down-weighted by `history_weight`), the base allele frequencies are reused so relationships

stay comparable, and a new set is chosen among the lines not yet phenotyped. Any other `tso_design()` argument (targets, budget, quotas) can be passed as well. Suppose next season's targets are the lines L201 to L250 that were not phenotyped this season:

```
new_targets <- setdiff(ids[201:250], plan$selected$id)
next_season <- tso_update(plan, data = tp, n = 40, target = new_targets,
                        history_weight = 0.8, verbose = FALSE)

next_season
#> TSOpt plan · inbred · 40 lines of 219 candidates · 21 named targets
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8)
#> ↳ Search greedy (robust CDmean: average over h2) (0.1 s)
#> Expected CDmean 0.327 (at h2 = 0.50) (exact, intercept projected: 0.327) vs
#> ↳ random 0.187 ± 0.014 (better than 100% of 100 random sets)
#> Verdict GREEN optimisation pays off here: expected accuracy ~32.4% above random
#> Structure discrete groups (4 groups, silhouette 0.65) -> stratified allocation
#> ↳ 0/0/0/40
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
#> ↳ unreachable (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
#> ↳ tso_plot(plan, "divide")
#> Alleles 100.0% of target alleles present in training (0 markers not covered) ·
#> ↳ 98.5% of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.36% to 2.23% of the CDmean (median 0.95%) · see
#> ↳ tso_plot(plan, "value")
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
no_history <- tso_design(tp, n = 40, target = new_targets,
                      exclude = plan$selected$id, verbose = FALSE)
round(c(with_history = next_season$value$cdmean,
        without_history = no_history$value$cdmean), 3)
#> with_history without_history
#> 0.327 0.283
```

What to look at: none of the 60 lines phenotyped this season is chosen again (0 overlap). Counting this season as history lifts the expected CDmean of the same 40 new plots from 0.283 to 0.327. Last season's targets, now unphenotyped candidates, can be chosen too.

12 Which crosses to make: `tso_crosses()`

A cross is worth its best progeny, not its mean. `tso_crosses()` ranks crosses among candidate parents by their **usefulness**, the progeny mean plus the selection intensity times the progeny genetic SD. The SD comes from the parents' differences at every marker, the marker effects and the recombination between markers (Haldane map; give `cM_per_Mb` when the map has no cM positions):

```
par <- names(sort(y, decreasing = TRUE))[1:12]
cr <- tso_crosses(tp, parents = par, pheno = y, cM_per_Mb = 1)
cr
#> TSOpt cross prediction · 66 crosses among 12 parents · DH progeny · usefulness =
#> ↳ mean + 1.75 x SD (top 10%)
#> marker effects estimated by rrBLUP: variances are shrunk; use the ranking
#> rank parent 1 parent 2 mean SD usefulness
#> 1 L241 L120 2.167 0.325 2.738
#> 2 L010 L120 2.153 0.326 2.726
#> 3 L167 L120 2.177 0.287 2.681
#> 4 L241 L103 1.846 0.452 2.639
#> 5 L241 L010 1.957 0.366 2.600
#> 6 L129 L120 2.169 0.239 2.589
```

```
#> 7      L010      L129      1.959      0.353      2.580
#> 8      L286      L167      1.950      0.354      2.572
#> 9      L241      L286      1.940      0.358      2.569
#> 10    L255      L241      1.973      0.339      2.568
#> 6 of the top 10 crosses by usefulness are not in the top 10 by mean alone.
```

The predicted progeny SD matches simulated doubled haploids within 1-2% (see the tests). To predict crosses well, design the training set for the candidate parents: `tso_design(tp, n = 60, target = par)`.

13 Which parents, and how much of each

After prediction comes crossing. Taking the top lines by GEBV (truncation selection) gives the most gain now, but uses up genetic diversity quickly. **Optimum contribution selection** (Meuwissen 1997) chooses the parents *and* how much each contributes, so that the expected genetic value of the offspring is as high as possible while the loss of diversity is held at an agreed rate: `delta_F`, the rate of inbreeding per generation (0.01 corresponds to an effective population size of 50).

`tso_contribution()` takes the breeding values from `tso_evaluate()` (here the GEBVs of the 50 targets, this season's new lines) and the same relationship matrix:

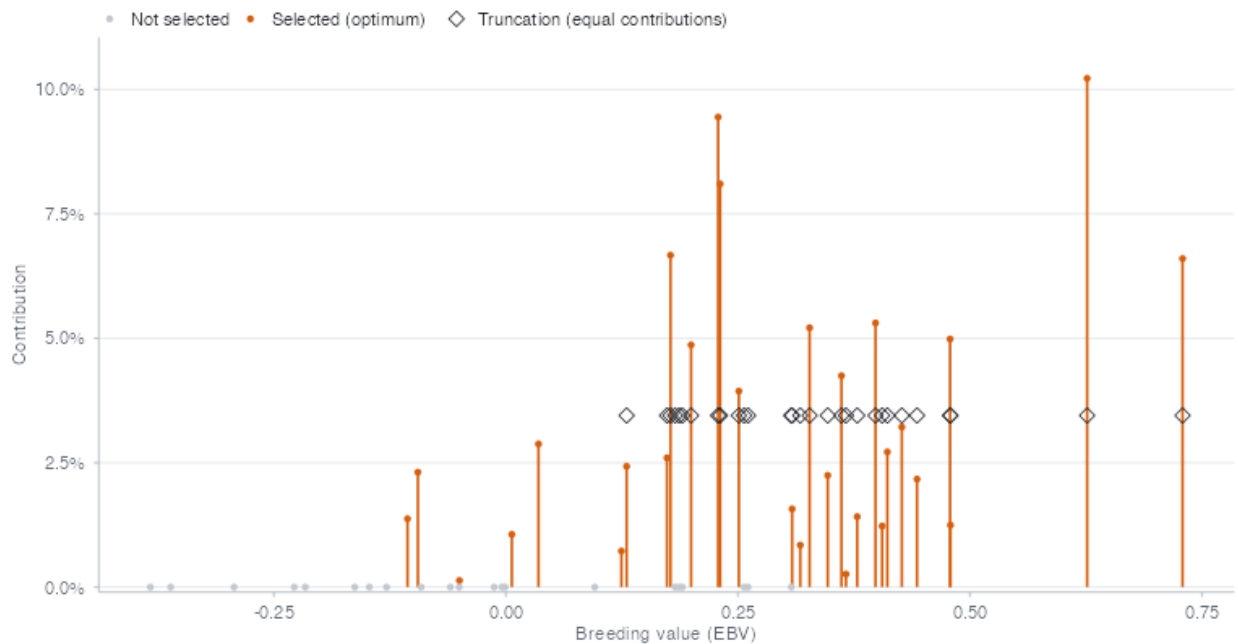
```
ebv <- setNames(ev_y$predictions$predicted, ev_y$predictions$id)
oc <- tso_contribution(ebv, tp$K_lines[names(ebv), names(ebv)],
                      delta_F = 0.01, n_matings = 20)

oc
#> TSOpt optimum contributions · 50 candidates ·  $\Delta F \leq 0.01 \rightarrow$  group coancestry  $\leq 0.1064$ 
#> Optimum      29 parents · mean EBV of offspring 0.334 · group coancestry 0.1064 ·
  ↪ expected  $\Delta F$  0.0100
#> Truncation    top 29 by EBV, equal contributions · mean EBV 0.330 · group
  ↪ coancestry 0.1197 ·  $\Delta F$  0.0246
#> Status        coancestry limit binding ·  $\lambda = 6.186$ 
#> Parents       L295 10.2% (4), L275 9.4% (4), L281 8.1% (3), L283 6.7% (3), L300
  ↪ 6.6% (3), L277 5.3% (2), L298 5.2% (2), L265 5.0% (2), L251 4.9% (2), L297 4.2%
  ↪ (2), ... (19 more)
```

```
tso_plot(oc)
```

Optimum contributions: 29 parents, mean EBV 0.33 vs 0.33 by truncation

Group coancestry 0.1064 (limit 0.1064) vs 0.1197 for the top 29 with equal contributions



What to look at: the Optimum and Truncation lines. With the same number of parents (29), optimum contributions give a mean EBV of 0.334 at the agreed rate of inbreeding (0.01); truncation with equal contributions gives 0.33 but inbreeds at 0.025 per generation. Almost the same gain for less than half the loss of diversity. With `n_matings`, the `n_matings` column of `oc$contributions` gives the number of crosses per parent.

14 The long term

Most comparisons of training-set methods look at one cycle of accuracy. `tso_simulate()` runs recurrent genomic selection for several cycles under each strategy and tracks genetic gain, genetic variance, accuracy and favourable alleles lost. All strategies start from the same founders in each replicate, so differences are due to the strategy alone. The settings below are small so the guide knits fast.

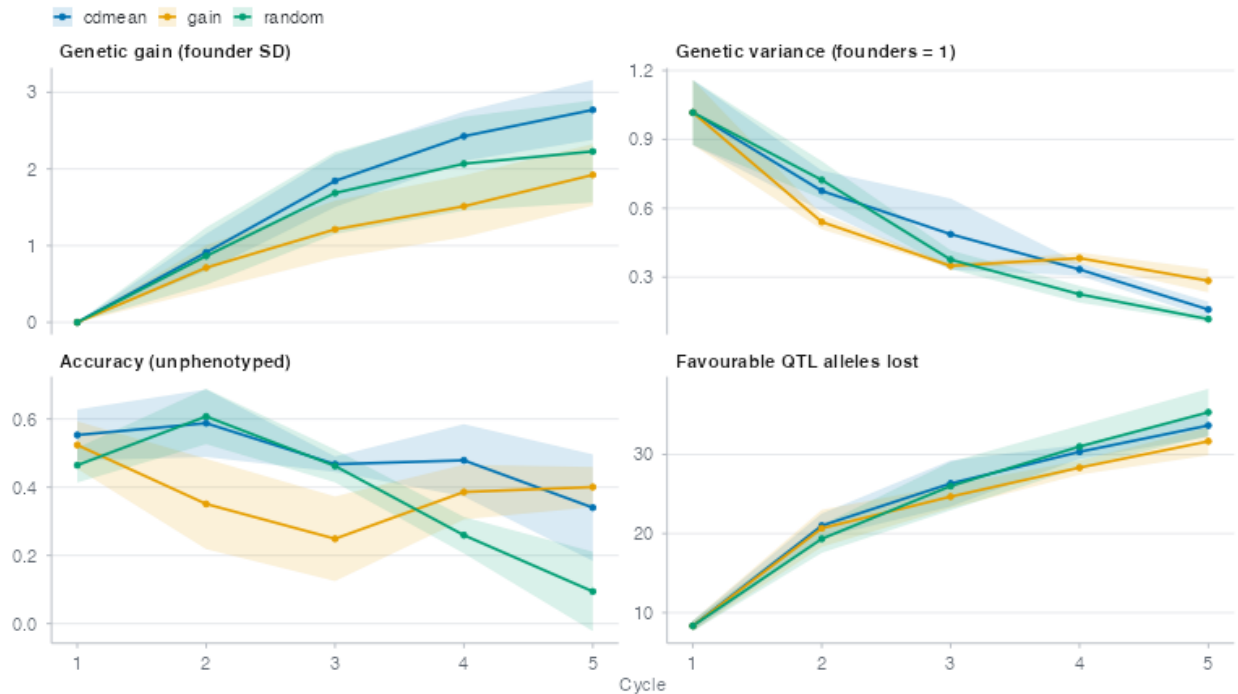
```
sim <- tso_simulate(strategies = list(random = "random", cdmean = "cdmean",
                                     gain = "gain"),
                  cycles = 5, reps = 3, n_candidates = 120, n_crosses = 12,
                  n_train = 40, n_parents = 12, markers = 500, n_qtl = 100,
                  seed = 7, verbose = FALSE)
```

```
sim
#> TSOpt long-term simulation · 5 cycles · 3 replicates · 40 of 120 candidates
#> ↳ phenotyped, 12 parents
#> Strategy Gain (founder SD) Genetic variance (vs founders) Mean accuracy
#> cdmean 2.77 0.16 0.49
#> gain 1.92 0.28 0.38
#> random 2.23 0.12 0.38
#> Favourable QTL lost Coancestry of parents
#> 33.7 0.86
#> 31.7 0.82
#> 35.3 0.93
#> Values at the last cycle, averaged over replicates (accuracy averaged over cycles).
```

```
tso_plot(sim)
```

Long-term consequences of the training-set strategy

3 replicates · mean $\pm$ 1 SE · 40 of 120 phenotyped per cycle



```
last <- subset(sim$results, cycle == max(cycle))
g_last <- tapply(last$gain, last$strategy, mean)
round(g_last, 2)
#> cdmean gain random
#> 2.77 1.92 2.23
```

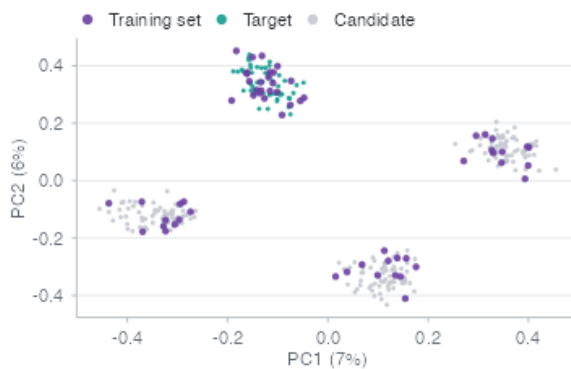
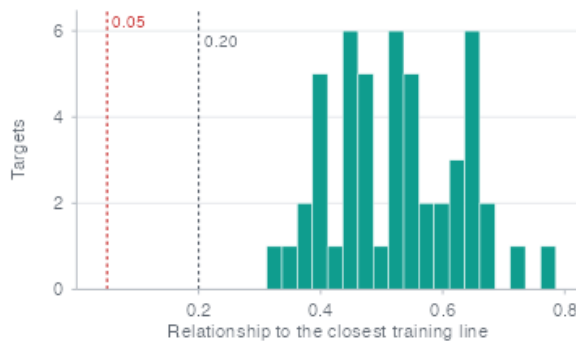
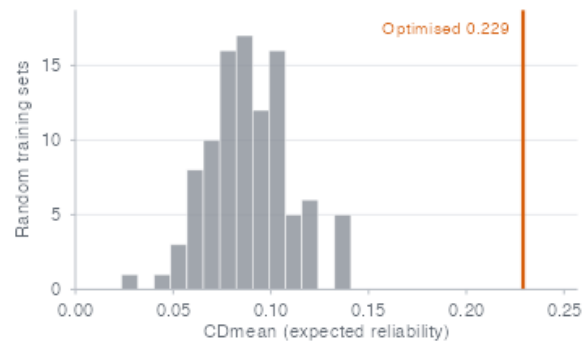
What to look at: in this small run the cdmean strategy reaches 2.77 founder SD of gain after 5 cycles, against 2.23 for random training sets and 1.92 for gain; also compare the genetic variance kept and the favourable alleles lost. Three replicates of a small programme are noisy. Before drawing conclusions, run the defaults (6 cycles, larger populations) with 10 or more replicates, and the settings of your own programme.

15 Figures for papers, slides and reports

Every figure is an ordinary ggplot2 object with one colour per role, used in every figure: training purple, targets teal, random grey, optimised orange. The gallery for a plan is `tso_plot(plan, type)` with type one of "summary" (four panels), "pca", "divide", "coverage", "baseline", "search", "kinship" and "value", plus "hybrid_grid", "met_allocation", "trait_allocation" and "major" for the special designs. Results of the other functions (comparison, uncertainty, economics, evaluation, top-k, Pareto front, batch, contributions, simulation, and from guide 3 layout, trait value, genotyping set and transfer) plot with `tso_plot(x)` or `plot(x)`.

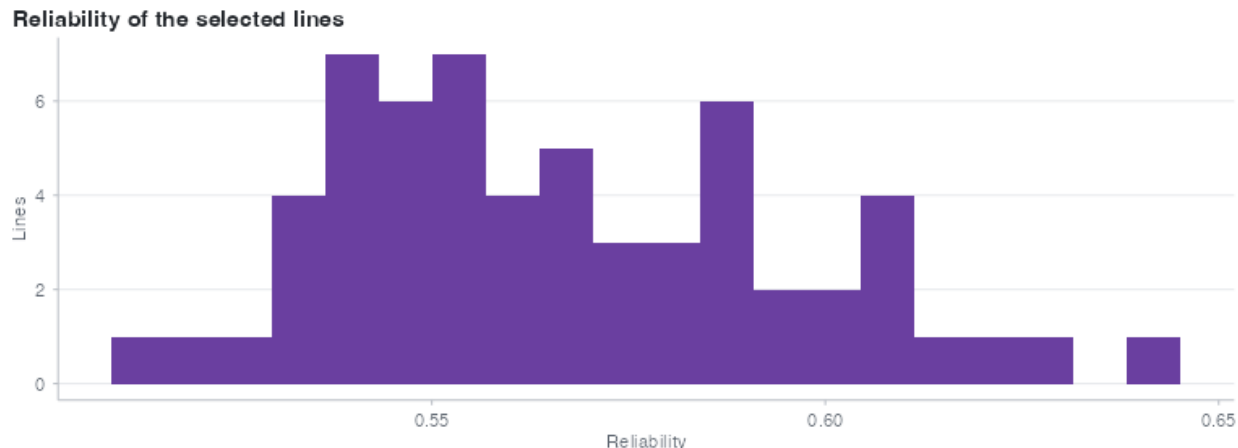
```
tso_plot(plan)
```

Training-set plan: 60 units · GREEN

A Genetic space
60 chosen**B Reliability per candidate**
None in the unselectable zone ($r^2 < 0.06$)**C Closest training relative**
100% of targets ≥ 0.20 **D Optimised vs random**
Better than 100% of random sets

`tso_colors()` and `tso_theme()` put your own figures in the same style, and `style = "presentation"` enlarges text for slides ("course" gives the PlantNura teaching style).

```
library(ggplot2)
tso_colors()
#> training target candidate history random optimised warning other
#> "#6A3FA0" "#0F9D8E" "#C4C8CF" "#E3A33B" "#8A9099" "#D65F0E" "#C62828" "#E4E6EA"
#> ink muted
#> "#1F2328" "#5E6670"
ggplot(plan$selected, aes(reliability)) +
  geom_histogram(bins = 20, fill = tso_colors()[["training"]]) +
  tso_theme() +
  labs(title = "Reliability of the selected lines", x = "Reliability",
        y = "Lines")
```



`tso_save_figure()` writes a figure at the exact column width a journal asks for, with fonts embedded and 600 dpi for raster formats. Presets: generic, nature, plos, g3, genetics, tag, springer, frontiers, plant_genome, crop_science and elsevier; width is "single", "onehalf", "double" or a number in mm.

```
out <- tempdir()
f <- tso_save_figure(tso_plot(plan, "pca"), file.path(out, "Figure2"),
                    journal = "g3", width = "single", format = c("pdf", "png"))
basename(f)
#> [1] "Figure2.pdf" "Figure2.png"
```

16 The field list and the one-page report

`tso_export()` writes the selected lines (or what = "all" lines with their role and expected reliability) as CSV, for the field book. `tso_report()` writes a self-contained HTML page for the programme manager: the decision, the expected gain over random, the risks and the figures. It needs neither pandoc nor LaTeX.

```
field <- tso_export(plan, file.path(out, "trial_2026.csv"))
head(field, 3)
#>   order  id line cost reliability forced
#> 1     1  L234 L234    1  0.5875783 FALSE
#> 2     2  L226 L226    1  0.5800052 FALSE
#> 3     3  L235 L235    1  0.5886286 FALSE
rep_file <- tso_report(plan, file.path(out, "plan_2026.html"), comparison = cmp,
                      title = "Training set 2026", author = "Wheat programme")
basename(rep_file)
#> [1] "plan_2026.html"
```

17 Outside R scripts: the command line

The `tsopt` command-line tool runs the main steps without writing R, for pipelines and colleagues who work in a terminal:

```
tsopt design --geno panel.vcf.gz --n 150 --out plan.json
tsopt diagnose --geno panel.vcf.gz --n 150
```

Its location is `system.file("cli", "tsopt", package = "TSOpt")`; `tso_cli("help")` lists every command.

18 Decision checklist

Before the plan goes to the field:

1. **Would it have worked before?** `tso_backtest()` on your programme's history.
2. **Is optimisation worth it here?** Check the verdict and `tso_diagnose()` (guide 2). AMBER or RED: a simple stratified sample may do.
3. **Are the targets right?** A targeted plan is worth far more than an untargeted one (guide 3).
4. **Is field reality in?** Checks in `must_include`, lost seed in `exclude`, last season in history.
5. **Does it beat what you would have done?** `tso_compare()` with your own list in others.
6. **Is the advantage real?** `tso_uncertainty()`: the verdict should be robust, or at least small, not uncertain.
7. **Can it find the best lines?** `tso_topk()` against random sets.
8. **Are major genes covered?** The Alleles line of the plan; guide 4 if a known gene matters.
9. **Is diversity acceptable?** `tso_pareto()` if coancestry is a concern.
10. **Is the size right?** `tso_sample_size()` (guide 2): stop where extra lines buy little; `tso_economics()` with your own prices: stop where the last plot no longer pays for itself.
11. **Robust to what you do not know?** Heritability (default), trait parameters (`trait_uncertainty`, default) and gene action (kernels), guide 3.
12. **Is the field plan ready?** `tso_layout()` (guide 3); `plan$selected$value` says which plots can go if seed or land runs short.

After harvest:

13. **How good was it?** `tso_evaluate()` with the heritability passed as `h2`, and `forward = TRUE` for predictions into a new season.
14. **Where do the next plots go?** `tso_next_batch()` within the season, `tso_update()` for the next one.
15. **Which parents?** `tso_contribution()` with an agreed `delta_F`, compared with truncation.
16. **Does the strategy hold up?** `tso_simulate()` with your programme's settings.
17. **Record it.** `tso_export()` for the field team, `tso_report()` for the manager, `tso_save_figure()` for the paper.